

OSInsight AI Project Report

AI Powered Performance Analyzer for Operating System Processes

Certificate

This certifies that OSInsight-AI was developed as a university project on AI-powered operating system performance analysis. The submitted work contains the implemented monitoring application, source code, tests, diagrams, and supporting artifacts.

Acknowledgement

The project acknowledges the open-source Python ecosystem and the maintainers of psutil, pandas, scikit-learn, CustomTkinter, and ReportLab. Their tools make reproducible systems analysis practical for student projects.

Abstract

OSInsight-AI monitors CPU, memory, disk, and process activity on a local machine. It stores observations in CSV form, forecasts short-term CPU and memory usage with an explainable linear regression model, detects pressure conditions, and presents recommendations through a desktop dashboard. The design favors transparent analytics and local-first handling of telemetry.

Introduction

Modern operating systems schedule many competing processes. Users often see a slow system without a concise way to relate the slowdown to resource use. This project provides a compact monitoring and analysis layer that turns raw process and hardware readings into actionable evidence.

Problem Statement

Conventional task lists expose instantaneous values but do not forecast likely pressure or summarize system health. A user needs a local tool that combines observation, trend analysis, process ranking, and report generation.

Project Objectives

The objectives are to sample real-time resource usage, retain historical observations, forecast CPU and memory demand, identify bottlenecks, recommend safe actions, present a live dashboard, and export evidence for later review.

Scope

The current version runs on one host and reads telemetry through psutil. It does not terminate processes, alter system settings, or transmit data. Forecasts represent a near-term linear trend and should support, rather than replace, administrator judgment.

Module Wise Breakdown

The monitoring module creates SystemSnapshot records from CPU, memory, disk, and process readings. The analytics module cleans history, fits a linear trend, calculates health, and applies threshold rules. The dashboard and reporting module presents live data and exports CSV or PDF summaries.

Technology Used

Python 3.12+ provides the implementation language. CustomTkinter provides the desktop interface. psutil supplies operating-system metrics. pandas and NumPy manage observations. scikit-learn implements the regression estimator. ReportLab creates exported PDF reports. pytest executes automated checks.

System Architecture

SystemMonitor is isolated from the rest of the application so platform access remains easy to test and replace. DataLogger serializes snapshots. AnalyticsEngine consumes the same tabular schema and returns a small immutable result. The dashboard renders that result and invokes exporters. The editable Mermaid architecture diagram is available in `diagrams/system_architecture.mmd`.

Flow Diagrams

Six Mermaid diagrams accompany the project: system architecture, data flow, application workflow, AI prediction pipeline, monitoring workflow, and report generation workflow. They are kept as source files in diagrams so they can be reviewed directly on GitHub.

Implementation Details

Sampling is non-blocking and ranks available process records by CPU and memory percentage. The dashboard retains up to 500 samples in memory while DataLogger persists the durable CSV record. Each refresh updates health indicators, forecasts, and a top-process text view.

Algorithms Used

For each resource series, the model assigns sequential indices to recent samples and fits ordinary least squares linear regression. It projects one index ahead and clips the forecast to the 0 to 100 percent range. Threshold rules flag values at or above 80 percent, while a weighted CPU-memory-disk expression produces the health score.

AI Model Explanation

Linear regression is suitable for an explainable baseline: the next estimate follows the observed local trend and needs little data. The model becomes more useful after several dashboard refreshes. Future work can compare this baseline with tree-based or recurrent methods using longer histories.

Testing

Automated tests cover empty-history handling, rising-CPU bottleneck detection, and removal of invalid telemetry values. Manual acceptance checks cover the live dashboard, CSV export, and PDF export. Test cases are recorded in `documentation/TEST_CASES.md`.

Results

With the supplied sample history, the analyzer identifies an increasing CPU and memory trajectory while disk use remains stable. The dashboard presents current resource values together with forecasts and ranked processes. Results vary by host because the application uses live local telemetry.

Screenshots

The application intentionally avoids embedding machine-specific screenshots in the submission report. Run the dashboard on the evaluation machine and store approved captures under screenshots/. The dashboard consists of the title, health status, four metric cards, ranked processes, and export controls.

GitHub Revision Tracking

The repository uses focused commits for monitoring, analytics, dashboard and exports, documentation, generated artifacts, and validation. Git history records incremental development and the remote repository remains the collaboration source of truth.

Challenges

Process access may be denied by the operating system, so the monitor skips inaccessible processes. The first CPU percentage sample can be low because psutil primes a non-blocking counter. Forecast quality depends on the length and representativeness of retained history.

Conclusion

OSInsight-AI delivers a usable local monitor with explainable forecasting rather than only a static process list. The implementation joins real-time observation, trend assessment, bottleneck guidance, testing, exports, and documentation in one submission-ready repository.

Future Scope

Future releases can add long-lived per-process histories, configurable alerts, trained anomaly detection, an authenticated fleet view, model persistence, and access-controlled remote aggregation.

References

psutil documentation. pandas documentation. scikit-learn LinearRegression documentation. CustomTkinter documentation. ReportLab user guide. Each dependency is listed in requirements.txt for reproducible installation.

Appendix Source Code

Complete source code is included in the repository. Key implementation files are `monitoring/system_monitor.py`, `analytics/engine.py`, `gui/dashboard.py`, `utils/reporting.py`, and `main.py`. Automated tests are stored under `tests/`.